Machine Learning Techniques

Unit - 5 [One Shot]

Most important topics:

1. Reinforcement learning, tasks & examples
2. Markov Decision Process
3. Q-Learning (functions, algorithm)
4. Deep Q-learning (introduction)
5. Genetic Algorithm

Courselearning

① Reinforcement Learning:

- It is a type of learning in ML where an agent learns to make decision by taking actions in an environment to receive feedbacks as reward or penalty.

* Key components:

- Agent: Learner or decision-maker.
- Environment: External system through which the agent interacts.
- State (S): A representation of current situation of agent.
- Action (A): Choices available to the agent.
- Reward (R): Immediate positive feedback from environment after an action.
- Penalty (P): Immediate negative feedback.
- Policy (π): Strategy that agent follows to determine its actions.

* Tasks in Reinforcement Learning:

- Exploration and exploitation: The agent must explore the environment to find the best actions and exploit known actions that provide high rewards.
- Learning from interaction: The agent learns by continuously interacting with the environment and updating its strategy based on

* Examples:

- Game playing: like chess is played by agents that learn strategies using Reinforcement Learning.
- Robotics: RL is used by robots to learn tasks like walking, navigating, or manipulating objects.

@brewilearning

(iii) Markov Decision Process (MDP):

- It is a mathematical framework for modelling decision-making situations where outcomes are partly random and partly under the control of the agent.

* Key components:

- Sab States (s): Set of all possible situations in which the agent can be.
- Actions (a): Set of all possible actions an agent can take
- Transition model (T): The probability of transitioning from one state to another due to action a.
- Reward function (R): The immediate reward received after transition.
- Policy (π): Defines agent's behaviour.

* MDP framework:

1. The agent starts in a state 's'.
2. It chooses an action 'a' based on it's policy 'π'.
3. It receives a reward R(s,a) and transitions to a new state 's'' according to a transition model T(s, a, s').
4. The process repeats, and the agent aims to maximize the cummulative reward over time.

@brewilearning

Q-Learning:

- It is a popular, model-free Reinforcement learning algorithm that learns the quality of actions, denoted as Q-values, which represents the expected utility of taking an action in a given state and following the optimal policy.

- Q-Function (Q):

- The Q-value $Q(s,a)$ represents the expected cumulative reward of taking action 'a' in state 's' following the optimal policy.

- The goal is to learn the optimal Q-function $Q^*(s,a)$ which gives the highest expected reward for any state-action pair.

* Algorithm:

- Initialize the Q-table $Q(s,a)$.

- For each episode:

  - Initialize the starting state 's'.

  - Choose an action 'a' using an exploration strategy.

  - Take action 'a' and observe the reward 'r' and the next state 's'.

  - Update the Q-value using the Bellman equation:

$$Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$

where,

α: learning rate, control the amount of updation

γ: discount factor, determines importance of future rewards

  - Update the state s to the new state s'.

SUBSCRIBE → @brevitearning

Deep Q-Learning :

- It is an extension of Q-Learning that uses a neural network to approximate the Q-values, making it feasible to handle large or continuous state spaces.

Introduction:

- Neural network: Instead of a Q-table, DQL uses a neural network to approximate the Q-values.
- Experience replay: To stabilize training, DQL stores the agents experiences in a replay buffer.
- Target Network: DQL uses two neural networks: the primary network Q and the target network Q'. It helps in stablizing the training by providing consistent targets during updates.

@brainlearning

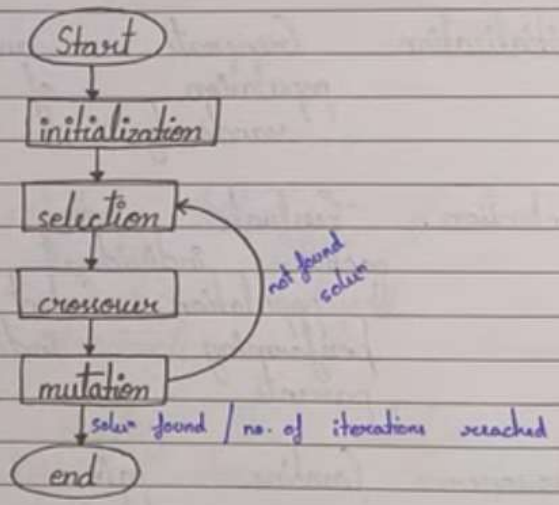
Genetic Algorithm.

- It is an optimization technique based on the principles of natural selection and genetics.

Steps:

1. Initialization: Generate an initial population of candidate randomly.
2. Selection: Evaluate the fitness of each individual in the population. Select best performing individuals to be parents.
3. Crossover: Combine pairs of parents to create new offspring.
4. Mutation: Introduce random changes to some offspring to maintain genetic diversity and explore new parts of the solution space.
5. Replacement: Replace the old population with the new generation of idividuals.

6. Iteration: Repeat the selection, crossover, mutation, and replacement steps for a no of generations or until a solution is found.



Thanks for watching !!

@bruvilearning